PROJECT REPORT · PROPOSED EXTENSION

MovieSense

Sentence-Based Movie Genre Classification and Cast Recommendation Using NLP

An NLP extension to the project "Movie Genre Classification and Cast Recognition using Deep Learning"

SUBMITTED BY

Team Number	**28**
Slot	**G2+TG2**
Name	**Poojikasri V.**
Registration Number	**24BDS0139**

Document type: design proposal. The dataset has not yet been acquired and no model has been trained, so this report contains **no experimental results**. All scores and outputs shown are illustrative examples.

October 2026

Contents

Abstract . . . 4
1. Introduction and Background . . . 4
2. Existing System and Its Limitations . . . 5
2.1 Input, output and architecture . . . 5
2.2 Limitations relevant to the proposed work . . . 5
3. Problem Statement . . . 5
4. Research Objectives . . . 6
5. Proposed Contribution and Novelty . . . 6
5.1 Inherited, new and extended modules . . . 6
6. Literature-Informed Justification . . . 7
7. Dataset Description and Preparation Strategy . . . 8
7.1 Contents and relevance . . . 8
7.2 Files required . . . 8
7.3 Joining summaries and genre labels . . . 9
7.4 Missing values, duplicates and inconsistencies . . . 9
7.5 Multi-label target encoding . . . 9
7.6 Training, validation and test splits . . . 10
7.7 Measures to avoid data leakage . . . 10
7.8 Train–inference mismatch: summaries versus sentences . . . 10
7.9 Limitations, licensing and usage . . . 10
8. Proposed System Architecture . . . 11
9. Detailed Explanation of Architecture Components . . . 12
9.1 Offline training pipeline (Panel A) . . . 12
9.2 Online inference pipeline (Panel B) . . . 12
9.3 Why cast is retrieved and never predicted . . . 13
9.4 Explanation module . . . 13
10. Methodology and Algorithm Workflow . . . 13
10.1 Training algorithm . . . 13
10.2 Inference algorithm . . . 13
10.3 Key design choices and trade-offs . . . 14
11. Illustrative Input and Output . . . 14
12. Implementation Plan and Technology Stack . . . 15
12.1 Implementation milestones . . . 15
12.2 Suggested project structure . . . 16
13. Evaluation Methodology and Comparison Metrics . . . 16
13.1 Classification metrics . . . 16

13.2 Baseline and fair comparison protocol . . . 16
13.3 Retrieval and explanation metrics . . . 17
13.4 Comparison with the existing poster branch . . . 17
14. Expected Benefits and Potential Applications . . . 17
15. Limitations, Ethical Considerations and Future Scope . . . 17
15.1 Limitations . . . 17
15.2 Ethical and security considerations . . . 18
15.3 Future scope . . . 18
16. Conclusion . . . 18
References . . . 18

Status of this work and provenance labels

This report is a **design proposal**. The CMU Movie Summary Corpus has **not** yet been downloaded or processed, no model has been trained, and no experimental results exist. Every score and output example in this report is illustrative. Text is labelled as follows where the distinction matters: **[Existing]** describes the existing poster-based project, **[Proposed]** is a design decision made in this report, **[Illustrative]** is an example and not a finding, and **[Planned]** is future experimental work. Dataset facts are taken from the corpus's official page and README [1].

Abstract

The existing project, *Movie Genre Classification and Cast Recognition using Deep Learning*, takes a movie poster as input. It predicts multiple genres with a convolutional neural network and identifies cast members through face detection and face recognition. This report proposes **MovieSense**, a complementary natural-language pipeline whose only required input is a single plot sentence or short synopsis. A pretrained transformer encoder (DistilBERT [3]) is fine-tuned with a multi-label classification head to output a confidence score for every genre. The same encoder embeds the plot summaries of a large public dataset, the CMU Movie Summary Corpus (42,306 plot summaries with aligned movie and character metadata [1]), into a vector index. That index supports similar-movie retrieval and *evidence-based* cast retrieval. Cast names are returned only when they come from a matched metadata record and are never inferred from the sentence alone. Token-level attributions (Integrated Gradients [7]) show which words drove each genre score. The report describes the dataset preparation strategy, a training and inference architecture, a leakage-safe evaluation protocol against a TF-IDF baseline using micro-F1, macro-F1, precision and recall, and the limitations and ethical considerations of the design. No accuracy claims are made. All improvements are framed as hypotheses to be tested.

1. Introduction and Background

Movie genre is a multi-label property: a single film can be a crime story, a mystery and a thriller at once. Automatic genre assignment supports cataloguing, search, recommendation and content analysis. Signals for genre exist in several modalities. Posters encode genre through colour, typography and composition. Plot text encodes it through events, characters and setting.

The existing project explores the visual modality. Its faculty-directed extension asks for a new contribution that (a) uses a sentence as input, (b) is trained on a large dataset, (c) is described by a new architecture diagram, and (d) demonstrates the expected output clearly. Transformer encoders pretrained on large text corpora, such as BERT [2] and its compressed variant DistilBERT [3], are a standard choice for sentence-level classification. They produce contextual representations that can be fine-tuned for a downstream task with modest labelled data. Libraries such as Hugging Face Transformers [4] make this practical within a student project.

MovieSense therefore adds a text branch next to the poster branch. It does not replace the poster branch. It answers a different question: "Given only a description of a story, what genres does it suggest, which known movies resemble it, and who appeared in those movies?"

2. Existing System and Its Limitations

> **Source of this section**
> The two reference documents for the existing project were not available while this report was prepared. This section therefore describes the existing system only at the level stated in the project brief: poster input, image preprocessing, CNN-based multi-label genre classification, face detection and face recognition. Specific architectures, datasets, accuracy figures and reference papers of the existing project are intentionally **not** stated here. The team should verify this section against the reference documents and insert their citations before submission.

2.1 Input, output and architecture [Existing]

Aspect	Existing poster-based system
Input	A movie poster image.
Preprocessing	Image preprocessing (for example resizing and normalisation) before the network.
Genre model	Convolutional neural network producing multi-label genre predictions.
Cast model	Face detection on the poster followed by face recognition of detected faces.
Output	Predicted genres for the poster and recognised cast members.
Principal techniques	Deep learning, CNNs, image preprocessing, face detection, face recognition.

Table 1. Summary of the existing system, as described in the project brief.

2.2 Limitations relevant to the proposed work

The points below are limitations *of the input modality* that motivate a text branch. They are not claims that the existing project is deficient in what it sets out to do.

- **A poster is required.** A user who knows only the story, such as a pitch, a half-remembered plot or an unreleased script, cannot query the system.
- **Posters are marketing artefacts.** They reflect design conventions and campaign choices. A poster's visual style may correlate imperfectly with the narrative content that defines genre.
- **Cast recognition depends on visible faces.** Recognition works only for faces printed on the poster and only for actors the recognition model can match. Text-only and illustrated posters provide no faces to recognise.
- **No narrative similarity.** An image-only pipeline cannot directly answer "which films have a similar story?", because story is not represented in its input.

3. Problem Statement

Given a free-form English sentence or short synopsis describing a movie plot, the system must:

1. Predict every applicable genre from a fixed genre set, with a calibrated confidence score per genre.
2. Retrieve the k most similar movies from a large plot-summary database.
3. Return cast details **only** for retrieved records that exist in the metadata. If the input is fictional or unmatched, the system must report cast information for the input as *unavailable* and must not generate names.

4. Indicate which words or phrases in the input most influenced each predicted genre.

The system must do this without requiring a poster image, using a model trained on a publicly documented dataset, and under an evaluation protocol that prevents data leakage.

4. Research Objectives

1. Design a sentence-to-genre multi-label classifier based on a pretrained transformer encoder.
2. Construct a clean, leakage-safe multi-label training set from the CMU Movie Summary Corpus.
3. Build a metadata store and a plot-embedding index for similar-movie search and evidence-based cast retrieval.
4. Provide token-level explanations of genre predictions.
5. Define a fair evaluation protocol comparing the transformer model with a TF-IDF baseline. Test the hypothesis that contextual embeddings improve macro-F1 on short, sentence-length inputs.
6. Compare the proposed text branch with the existing poster branch in terms of input, output and capability.

5. Proposed Contribution and Novelty

The contribution is the **sentence-based branch** and its retrieval layer. The individual techniques are established. The novelty for this project lies in their combination for a new input modality, in the evidence-based treatment of cast information, and in a training strategy that matches the short inference-time input.

1. **Natural-language plot input**: a sentence or synopsis replaces the poster as the required input.
2. **Semantic feature extraction**: DistilBERT produces contextual sentence embeddings.
3. **Multi-label genre prediction with confidence scores**: independent sigmoid outputs with per-genre thresholds tuned on validation data.
4. **Similar-movie retrieval**: cosine nearest-neighbour search over embedded plot summaries.
5. **Evidence-based cast retrieval**: cast is read from `character.metadata.tsv` for matched records only, with the matching record cited.
6. **Explainable predictions**: Integrated Gradients [7] attributions over input tokens.
7. **Existing-versus-proposed comparison**: a structured comparison (Table 3) and a protocol for a fair empirical comparison on a shared test set.

5.1 Inherited, new and extended modules

Category	Module	Notes
Inherited (concept)	Multi-label genre formulation	Independent per-genre probabilities with thresholds, as in the existing classifier.

Category	Module	Notes
Inherited (concept)	Cast as an output	The project goal of reporting cast is retained. The mechanism changes from face recognition to metadata lookup.
Inherited (concept)	Multi-label evaluation	The same family of metrics is used, so the branches can be compared.
New	Text preprocessing and validation	Unicode normalisation, length limits and input validation.
New	Transformer encoder + classification head	DistilBERT fine-tuned with BCE-with-logits loss.
New	Metadata store and vector index	Relational tables for movies, genres and cast, plus a plot-embedding index.
New	Similar-movie search	Top-k cosine neighbours with optional genre filtering.
New	Evidence-based cast retrieval	Cast is returned only from matched records, otherwise marked “unavailable”.
New	Explanation module	Token-level attributions per predicted genre.
Extended (future)	Late fusion with poster branch	Optional averaging of probabilities when both inputs exist.

Table 2. Which modules are inherited, new, or extended.

Dimension	Existing (poster)	Proposed (sentence)
Required input	Poster image	Plot sentence or synopsis
Signal used	Visual design, faces	Narrative content
Genre model	CNN, multi-label	DistilBERT + linear head, multi-label
Cast mechanism	Face detection + recognition on the poster	Lookup of stored cast for matched or similar records
Works without poster	No	Yes
Works for unreleased / fictional plots	Only if a poster exists	Yes for genres and similar movies; cast reported as unavailable
Similar-movie search	Not part of the described system	Yes, by plot similarity
Explanations	Not part of the described system	Token attributions per genre
Main failure mode	Misleading or text-only posters; faces not visible	Vague or very short sentences; domain shift from long summaries

Table 3. Existing versus proposed pipeline (qualitative).

6. Literature-Informed Justification

Scope of this section

The project brief asks that this justification be based on the reference documents of the existing project. Those documents were not available. The justification below therefore rests only on verifiable primary sources for the techniques used [1]–[9]. The team should add the reference papers cited in the existing project’s documents here.

- **Contextual encoders for classification.** BERT [2] introduced bidirectional pretraining and showed that fine-tuning one pretrained model with a small task head is effective across sentence-level tasks. That pattern is the one adopted here.

- **Compute-efficient encoder.** DistilBERT [3] is a distilled BERT that its authors report as 40% smaller and 60% faster while retaining 97% of BERT's language-understanding performance on their benchmarks. This makes fine-tuning on roughly 40,000 documents feasible on a single GPU.
- **Sentence embeddings for similarity.** Sentence-BERT [6] showed that raw BERT outputs are poor similarity vectors without suitable pooling or fine-tuning. MovieSense therefore treats retrieval quality as something to measure, and keeps a dedicated sentence-embedding model as a fallback option.
- **Attribution for explanation.** Integrated Gradients [7] satisfies sensitivity and implementation invariance, and needs only gradient access to the model. Captum [10] provides a maintained PyTorch implementation.
- **Multi-label stratification.** Random splits of rare, overlapping labels can leave genres missing from validation or test sets. Iterative stratification [9] is designed to preserve label proportions across splits.
- **Dataset.** The CMU Movie Summary Corpus [1] pairs Wikipedia plot summaries with Freebase movie- and character-level metadata. This pairing is exactly what genre labelling and cast retrieval require.

7. Dataset Description and Preparation Strategy

> **Data availability**
> **Planned dataset, not yet acquired.** All figures below come from the dataset's official README [1]. Counts after joining and cleaning are unknown until the preprocessing in Section 7.3 is run, and will be reported then.

7.1 Contents and relevance

The CMU Movie Summary Corpus was collected by Bamman, O'Connor and Smith at Carnegie Mellon University [1]. It contains 42,306 plot summaries extracted from English-language Wikipedia (November 2012 dump), plus metadata extracted from Freebase (November 2012 dump). The metadata covers 81,741 movies and 450,669 characters. It is relevant because it provides the three things MovieSense needs: natural-language plots (model input), genre labels (targets) and actor names linked to movies (cast evidence).

7.2 Files required

File	Used for	Key fields (from README [1])
plot_summaries.txt	Model input; retrieval corpus	Wikipedia movie ID, plot summary text
movie.metadata.tsv	Genre labels; titles; retrieval display	Wikipedia movie ID, Freebase movie ID, movie name, release date, box-office revenue, runtime, languages, countries, genres (Freebase ID:name tuples)
character.metadata.tsv	Evidence-based cast retrieval	Wikipedia movie ID, character name, actor name, plus actor attributes and Freebase IDs
corenlp_plot_summaries.tar.gz	Not required	CoreNLP-processed summaries (optional, 628 MB)

Table 4. Dataset files and their role in MovieSense.

Only the movie name, genres and Wikipedia ID are needed from the movie metadata, and only the actor name and character name from the character metadata. Actor attributes such as gender, height, ethnicity and date of birth are **deliberately not loaded**. They are not needed for the task, and storing them would add privacy and bias risk without benefit (data minimisation).

7.3 Joining summaries and genre labels [Proposed]

1. Parse `plot_summaries.txt` as (wiki_id, text) pairs. Each line holds the ID and then the summary.
2. Parse `movie.metadata.tsv`. Decode the genres column, which stores Freebase ID:name tuples, into a list of genre names.
3. Inner-join on Wikipedia movie ID. Summaries without metadata, or metadata rows without summaries, are dropped from the classification set. All metadata rows remain available to the metadata store.
4. Drop rows whose genre list is empty after label mapping (Section 7.5).

7.4 Missing values, duplicates and inconsistencies [Proposed]

Issue	Handling
Missing genres	Excluded from training, validation and test. Kept in the metadata store for title and cast lookup.
Missing or very short plots	Rows below a minimum token count (set after inspecting the length distribution) are excluded.
Missing release date	Not needed for classification. Shown as “unknown” in output.
Exact duplicate plot texts	Detected by hashing normalised text. One copy is kept.
Near-duplicates / remakes	Detected by embedding cosine similarity above a high threshold. They are placed in the same split group to prevent leakage.
Duplicate titles	Kept as separate records keyed by wiki_id, displayed with release year to disambiguate.
Inconsistent genre labels	Freebase genres are fine-grained and overlapping (for example several variants of “Action”). A documented mapping table merges synonyms into a coarse target set.
Missing actor names	Character rows without an actor name are not shown as cast.

Table 5. Data-quality handling.

7.5 Multi-label target encoding [Proposed]

The raw Freebase genre vocabulary is large and long-tailed. A coarse target set will be defined after exploratory analysis. Fine-grained labels are mapped to parent genres, and genres below a minimum support (for example a few hundred training examples) are dropped or merged. The expected result is roughly 15–25 genres such as Drama, Comedy, Thriller, Crime, Mystery, Romance and Horror. The exact set and counts will be reported once computed, not assumed. Each movie becomes a multi-hot vector $y \in \{0,1\}^{G}$ using scikit-learn’s `MultiLabelBinarizer`, fitted on the full label vocabulary so that column order is fixed and versioned.

7.6 Training, validation and test splits [Proposed]

A 70 / 15 / 15 split is planned, using iterative multi-label stratification [9] so that rare genres appear in every split. Stratification operates on *groups* rather than rows: a group is a normalised title plus its near-duplicate cluster, so remakes and duplicate entries never straddle splits. A fixed random seed and the list of IDs in each split are saved so experiments can be reproduced.

7.7 Measures to avoid data leakage [Proposed]

- Group-aware splitting, as described above.
- The TF-IDF vocabulary, label mapping statistics and decision thresholds are fitted on the training split (thresholds on validation) and never on the test split.
- The test split is used once, for the final reported numbers. Model selection uses validation only.
- During retrieval evaluation, the query movie is excluded from its own neighbour list.
- Plot summaries sometimes name the actors or the film itself. A check will measure how often title tokens appear in summaries, and title strings will be masked during training if they are found to be frequent, so the classifier cannot shortcut through memorised titles.

7.8 Train–inference mismatch: summaries versus sentences

Wikipedia plot summaries are often several paragraphs long, while MovieSense receives a single sentence at inference. A model trained only on full summaries (truncated to 512 tokens) may not transfer to one-sentence inputs. The design addresses this with **short-text augmentation**: each training example is also presented as randomly sampled windows of one to three sentences that inherit the movie's labels. Evaluation reports results separately for full-summary and single-sentence test conditions. The single-sentence condition (first sentence, and a random sentence) is the primary metric for this project.

7.9 Limitations, licensing and usage

- **Licence.** The corpus is released under a Creative Commons Attribution-ShareAlike licence [1]. Use requires attribution (citing [1]). Redistributed derivatives of the data, such as a cleaned copy, must carry the same licence.
- **Age and coverage.** The data reflect Wikipedia and Freebase as of November 2012. Films released afterwards are absent, and coverage is skewed toward English-language and widely documented films.
- **Label noise.** Freebase genres were crowd-curated and inconsistent. Some labels are missing or loosely assigned.
- **Summary ≠ sentence.** See Section 7.8.
- **Personal data.** Character metadata includes attributes of real people. Only names are used (Section 7.2).

8. Proposed System Architecture

Figure 1 shows the proposed architecture. Panel A is the offline training pipeline. Panel B is the online inference pipeline, which runs end-to-end from a sentence alone. Panel C is the existing poster-based branch, drawn dashed to show that it is a separate, optional system and not an input to Panel B. Its only connection to the new branch is an optional late-fusion step reserved for future work.

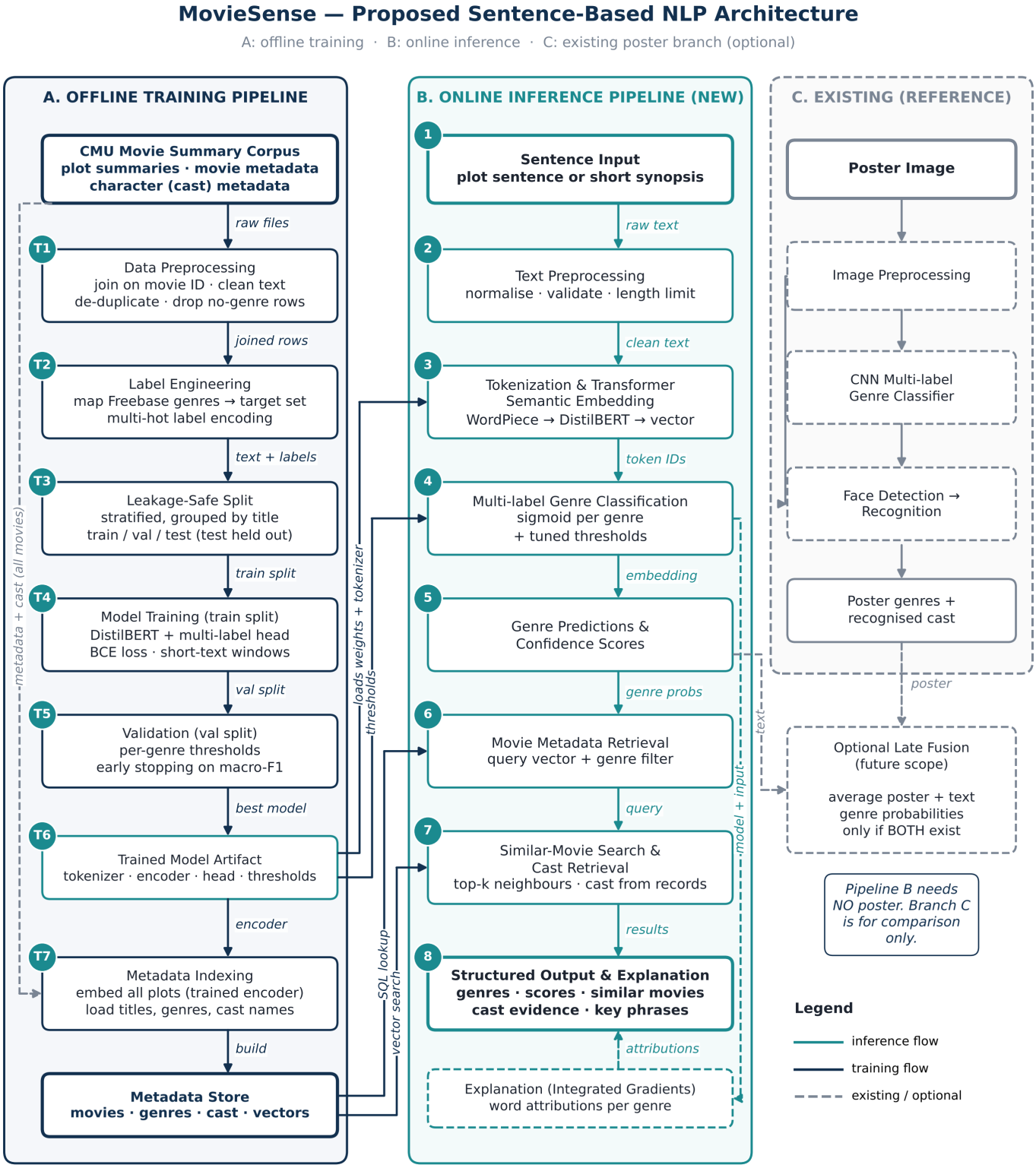


Figure 1. MovieSense architecture. Navy arrows carry training data and model artefacts. Teal arrows carry inference data. Grey dashed elements belong to the existing system or are optional.

9. Detailed Explanation of Architecture Components

9.1 Offline training pipeline (Panel A)

#	Component	Responsibility
T1	Data preprocessing	Parse the three files, join on Wikipedia movie ID, clean whitespace and markup, de-duplicate, and drop rows without genres.
T2	Label engineering	Apply the genre mapping table, prune rare genres, and produce multi-hot targets.
T3	Leakage-safe split	Group-aware iterative stratification into train/val/test. The test split is frozen.
T4	Model training	Fine-tune DistilBERT with a linear head on the train split using full summaries and short-text windows.
T5	Validation & calibration	Early stopping on validation macro-F1. Tune one decision threshold per genre on validation predictions.
T6	Trained model artefact	A versioned bundle of tokenizer, encoder weights, head, label list and thresholds.
T7	Metadata indexing	Embed every plot with the trained encoder. Load titles, genres and cast names into the store.
—	Metadata store	Relational tables (movies, genres, cast) plus a vector index of plot embeddings keyed by wiki_id.

Table 6. Training-pipeline components.

9.2 Online inference pipeline (Panel B)

#	Stage	Input → Output and design notes
1	Sentence input	A UTF-8 string from the user. This is the only required input.
2	Text preprocessing	Unicode NFC normalisation, control-character removal and whitespace collapse. Inputs are rejected if empty, if they exceed a maximum length (for example 2,000 characters), or if they contain no alphabetic content. Lower-casing is left to the tokenizer's own configuration.
3	Tokenization & embedding	WordPiece tokens with attention mask, truncated at 512 tokens. DistilBERT produces a 768-dimensional pooled vector (mean pooling over non-padding tokens).
4	Multi-label classification	A linear layer of size 768 × G, then a sigmoid per genre. Genre g is predicted when $p_g \geq \tau_g$, where τ_g is the tuned threshold. If no genre passes, the single highest-scoring genre is returned and flagged "low confidence".
5	Predictions & scores	Genres ranked by probability with scores, including near-threshold genres for transparency.
6	Metadata retrieval	Builds the retrieval query from the stage-3 vector and an optional filter on predicted genres.
7	Similar-movie & cast retrieval	Top-k cosine neighbours from the vector index. Cast rows are fetched by wiki_id from the store for each neighbour. An *input match* is declared only when cosine similarity $\geq \tau_{match}$ (set conservatively on validation data). Otherwise cast for the input is "unavailable".
8	Structured output & explanation	A JSON response containing genres, scores, neighbours with similarity, cast with source record IDs, and the top attributed tokens per genre from the explanation module.

Table 7. Inference-pipeline stages.

9.3 Why cast is retrieved and never predicted

A plot sentence carries no reliable information about which actors performed in a film. Two films with near-identical premises usually have entirely different casts, and a hypothetical plot has no cast at all. Generating actor names from text would therefore produce fabrications that look authoritative. MovieSense limits cast output to **facts stored in the metadata**, each tied to a source wiki_id. For neighbours, the output shows *"cast of similar movie X"*, never *"predicted cast of your plot"*. This is why the title says cast *recommendation*: the names are suggestions drawn from comparable films, not identifications.

9.4 Explanation module

For each predicted genre, Integrated Gradients [7] integrates the gradient of that genre's logit with respect to the input embeddings, along a path from a baseline (padding embeddings) to the actual input. The resulting token scores are aggregated to whole words and the top-scoring phrases are reported. Attention weights are not used as explanations, because they are not designed to measure feature importance. Attribution quality will be checked with a deletion test: removing the top-attributed words should reduce the genre score more than removing random words.

10. Methodology and Algorithm Workflow

10.1 Training algorithm [Proposed]

```
Input : plot_summaries, movie_metadata, genre_map, seed
Output: model bundle M, metadata store S

D  <- join(plot_summaries, movie_metadata, on=wiki_id)
D  <- clean(D); dedupe(D); D.genres <- map(D.genres, genre_map)
D  <- D[ len(D.genres) > 0 ];  Y <- MultiLabelBinarizer(D.genres)
groups <- title_and_near_duplicate_clusters(D)
train, val, test <- iterative_stratified_group_split(D, Y, groups, 0.70/0.15/0.15, seed)
enc <- DistilBERT(pretrained); head <- Linear(768, G)
for epoch in 1..E:
    for batch in augment_short_windows(train):        # full text + 1-3 sentence windows
        logits <- head(mean_pool(enc(tokenize(batch.text))))
        loss <- BCEWithLogits(logits, batch.y, pos_weight)
        update(enc, head, AdamW, linear warm-up schedule)
    if macroF1(val) not improved for P epochs: stop
tau_g <- argmax_t F1_g(val, t) for each genre g
M <- (tokenizer, enc, head, labels, tau); save with version + data hash
S <- build_store(all movies, cast names); index <- embed_all(plot_summaries, enc)
```

10.2 Inference algorithm [Proposed]

```
Input : sentence s, model M, store S, k
s  <- validate_and_normalise(s)              # reject empty / oversized input
v  <- mean_pool(M.enc(tokenize(s)))
p  <- sigmoid(M.head(v)); G* <- { g : p_g >= tau_g }  (fallback: argmax, flagged)
N  <- top_k_cosine(index, v, k, filter=G*)  # exclude exact self-match during evaluation
for n in N: n.cast <- S.cast(n.wiki_id)       # facts only, with source id
m  <- N[0] if sim(N[0]) >= tau_match else None
input_cast <- S.cast(m.wiki_id) if m else 'unavailable'
E  <- integrated_gradients(M, s, G*)          # top words per genre
return {genres: G*, scores: p, similar: N, input_cast, explanation: E}
```

10.3 Key design choices and trade-offs

Decision	Chosen	Alternative	Trade-off
Encoder	DistilBERT	BERT-base / RoBERTa	Lower accuracy ceiling in exchange for faster training and inference. Larger encoders are a planned ablation.
Loss	BCE + pos_weight	Focal loss	BCE is simple and stable. Focal loss may help rare genres and is a candidate ablation.
Thresholds	Per-genre, tuned on val	Global 0.5	Per-genre thresholds improve macro-F1 but can overfit a small validation set.
Retrieval vectors	Fine-tuned encoder	Sentence-BERT model	One model is simpler to run. A dedicated similarity model may retrieve better. Both will be measured.
Vector index	Exact cosine search	Approximate (FAISS [8])	About 42k vectors fit easily in memory for exact search. FAISS is reserved for scale-out.

Table 8. Design decisions.

11. Illustrative Input and Output

> **Illustrative only**
> Every number, title and attribution in this section is a hand-constructed **example of the output format**. None is produced by a trained model or a real query. Example titles were chosen by hand and have not been verified as present in the corpus or as true nearest neighbours.

Input sentence: *"A young detective investigates a series of mysterious murders while uncovering a conspiracy involving his closest friend."*

Genre	Illustrative confidence	Threshold (example)	Decision
Crime	0.88	0.42	Predicted
Mystery	0.81	0.38	Predicted
Thriller	0.74	0.40	Predicted
Drama	0.31	0.45	Not predicted (shown for transparency)
Action	0.14	0.40	Not predicted

Table 9. Illustrative genre scores. These are not experimental results.

Influential words (illustrative attributions).

- **Crime** ← "detective", "investigates", "murders"
- **Mystery** ← "mysterious", "investigates", "series of … murders"
- **Thriller** ← "conspiracy", "closest friend" (betrayal motif), "uncovering"

Similar movies and cast (illustrative format). Each neighbour lists cast only if its record exists in `character.metadata.tsv`. Names are filled from that file at run time and are deliberately not invented here.

Rank	Similar movie (example title)	Similarity	Cast shown
1	Se7en (1995)	0.71	From character.metadata.tsv for this wiki_id, if present

Rank	Similar movie (example title)	Similarity	Cast shown
2	Zodiac (2007)	0.68	From character.metadata.tsv for this wiki_id, if present
3	Memories of Murder (2003)	0.66	From character.metadata.tsv for this wiki_id, if present

Table 10. Illustrative retrieval output.

Cast for the input plot: *Unavailable*. The sentence is hypothetical, and no record exceeds the match threshold τ_match. A hypothetical plot cannot determine an actual cast, so the system reports this explicitly instead of generating names.

```
{
  "input": "A young detective investigates a series of mysterious murders ...",
  "genres": [ {"label": "Crime",    "score": 0.88},
              {"label": "Mystery",  "score": 0.81},
              {"label": "Thriller", "score": 0.74} ],
  "near_threshold": [ {"label": "Drama", "score": 0.31} ],
  "input_match": null,
  "input_cast": "unavailable: no matching record (max similarity below threshold)",
  "similar_movies": [ {"wiki_id": "<id>", "title": "<title>", "similarity": 0.71,
                       "cast": "<names from character.metadata.tsv>", "source": "CMU corpus"} ],
  "explanation": { "Crime": ["detective", "investigates", "murders"], ... },
  "illustrative": true
}
```

12. Implementation Plan and Technology Stack

Technology	Role	Why it is used
Python 3	Implementation language	The standard ecosystem for NLP and machine learning.
pandas [12]	Parsing, joining, cleaning	Tabular joins on wiki_id, missing-value handling and TSV parsing.
NumPy [11]	Arrays and vector maths	Embedding matrices, cosine similarity, threshold sweeps.
Hugging Face Transformers [4]	Tokenizer, DistilBERT, training loop	Pretrained checkpoints and a tested fine-tuning API.
PyTorch	Deep-learning backend	Required by Transformers and Captum.
scikit-learn [5]	Baseline and metrics	TF-IDF, one-vs-rest logistic regression, MultiLabelBinarizer, F1/precision/recall.
iterative-stratification	Multi-label splitting	Implements iterative stratification [9].
Captum [10]	Explanations	Integrated Gradients for PyTorch models.
SQLite → PostgreSQL	Metadata store	SQLite for a single-file prototype; PostgreSQL for a deployed service with concurrent reads.
FastAPI + Docker (optional)	Serving	Typed request validation, async endpoints and reproducible deployment.

Table 11. Technology stack.

12.1 Implementation milestones [Planned]

Phase	Deliverable	Exit criterion
1. Data	Parsed, joined, cleaned dataset; genre map; EDA notebook	Counts, label distribution and length distribution reported

Phase	Deliverable	Exit criterion
2. Splits	Group-aware stratified splits with saved IDs	Every target genre present in all splits
3. Baseline	TF-IDF + logistic regression	Validation metrics logged
4. Transformer	Fine-tuned DistilBERT with thresholds	Validation metrics logged; artefact versioned
5. Retrieval	Metadata store + vector index + cast lookup	Retrieval metrics on validation
6. Explanations	Integrated Gradients integration	Deletion test completed
7. Evaluation	Single run on the test split	Final tables with confidence intervals
8. Demo (optional)	API or notebook demo	End-to-end example from a sentence

Table 12. Milestones.

12.2 Suggested project structure

```
moviesense/
├── data/raw/            # original corpus files (not committed; CC BY-SA attribution kept)
├── data/processed/      # joined parquet, splits/*.txt (IDs only)
├── configs/             # genre_map.yaml, train.yaml
├── src/moviesense/
│   ├── data/            # loaders, cleaning, label mapping, splitting
│   ├── models/          # classifier, baseline, thresholds
│   ├── retrieval/       # embedding index, metadata store, cast lookup
│   ├── explain/         # integrated gradients wrapper
│   └── api/             # optional FastAPI service
├── notebooks/           # EDA and result analysis
└── tests/               # unit tests for parsing, leakage checks, schema
```

13. Evaluation Methodology and Comparison Metrics

13.1 Classification metrics

- **Micro-F1**: pools true and false positives across all genres. Dominated by frequent genres such as Drama.
- **Macro-F1**: the unweighted mean of per-genre F1. It treats rare genres equally and is the primary metric, because rare genres are the harder and more informative case.
- **Micro/macro precision and recall**: reported separately to show whether errors are false alarms or misses.
- **Per-genre F1** and **Hamming loss**: for error analysis.

13.2 Baseline and fair comparison protocol [Planned]

1. **Baseline:** TF-IDF (word 1–2-grams, vocabulary fitted on train only) with one-vs-rest logistic regression (scikit-learn [5]).
2. **Same data:** identical splits, label set and input text for both models, including the same short-text windows.
3. **Same tuning budget:** each model receives a comparable hyperparameter search on validation, and per-genre thresholds are tuned the same way for both.
4. **Same conditions:** both are evaluated on full-summary and single-sentence test inputs.

5. **Variance:** three random seeds per model, reported as mean ± standard deviation, with bootstrap 95% confidence intervals on test macro-F1.
6. **Hypothesis:** H1 states that DistilBERT achieves higher macro-F1 than TF-IDF on single-sentence inputs. H1 is accepted only if the confidence intervals of the difference exclude zero.

13.3 Retrieval and explanation metrics

- **Genre-overlap Precision@k**: the fraction of the top-k neighbours sharing at least one gold genre with the query (query excluded from its own results).
- **Mean Jaccard@k** between query and neighbour genre sets.
- **Cast-attribution precision**: on held-out real summaries, the fraction of returned "input match" records that are the correct film. This sets τ_match conservatively.
- **Explanation faithfulness**: the score drop after deleting top-attributed words versus random words.

13.4 Comparison with the existing poster branch

A numeric comparison between the branches is meaningful only on **the same films**. If posters are available for a subset of test-split movies, both branches will be evaluated on that intersection with a shared genre mapping. Otherwise the comparison remains the qualitative one in Table 3, and no accuracy ranking between branches will be claimed.

14. Expected Benefits and Potential Applications

The following are expected benefits, to be confirmed by the evaluation above:

- **Lower input barrier**: genre and similar-film suggestions from one typed sentence.
- **Narrative-level similarity**: retrieval by story rather than by visual style.
- **Trustworthy cast output**: every name is traceable to a source record. Unmatched inputs yield an explicit "unavailable".
- **Interpretability**: users and evaluators can see which words drove each genre.
- **Complementarity**: when a poster is also available, the two branches capture different signals (hypothesis for future fusion work).

Potential applications include script and pitch triage, catalogue tagging for archives and streaming libraries, "find a film like this story" search, and teaching material on multi-label NLP.

15. Limitations, Ethical Considerations and Future Scope

15.1 Limitations

- No results yet. Every performance statement in this report is a hypothesis.
- Genres are subjective, and Freebase labels are noisy. The ceiling on achievable F1 is unknown.
- One sentence may simply lack genre evidence. Such inputs should yield low-confidence outputs, not confident guesses.

- The corpus ends in 2012 and is English-centric, so recent and non-English cinema is under-represented.
- Retrieval quality from a classification-tuned encoder is uncertain (Section 6).

15.2 Ethical and security considerations

- **No fabricated identities**: the system never attributes a cast to an input it cannot match.
- **Data minimisation**: actor attributes such as ethnicity, height and date of birth are not loaded or stored.
- **Bias**: genre models can learn stereotypes, for example associating particular settings or groups with certain genres. Per-genre error analysis and inspection of attributions will be used to look for this.
- **Licence compliance**: CC BY-SA attribution is preserved in the code, the documentation and any derived data.
- **Input handling (if deployed)**: length limits and validation at the API boundary, parameterised SQL queries against the metadata store, rate limiting, and no logging of raw user text beyond what debugging requires.

15.3 Future scope

- Late fusion of poster and text probabilities when both inputs exist, evaluated on a shared test set.
- Larger or more recent encoders, and a dedicated sentence-embedding model for retrieval.
- Extending the corpus with newer, licence-compatible plot data.
- Multilingual input using a multilingual encoder.

16. Conclusion

MovieSense extends a poster-based genre and cast system with a text branch that needs only a plot sentence. The design pairs a fine-tuned DistilBERT multi-label classifier with a metadata store and plot-embedding index built from the CMU Movie Summary Corpus. This enables similar-movie search and cast information that is grounded in records rather than generated. The report defines the dataset preparation, a leakage-safe split, a remedy for the summary-versus-sentence mismatch, and a fair comparison against a TF-IDF baseline. The next step is to acquire the data, run the preprocessing, and test the stated hypotheses. The project's claims will be set by those results, not assumed in advance.

References

[1] D. Bamman, B. O'Connor and N. A. Smith, "Learning Latent Personas of Film Characters," in *Proc. 51st Annual Meeting of the Association for Computational Linguistics (ACL)*, Sofia, Bulgaria, 2013. Dataset: CMU Movie Summary Corpus, https://www.cs.cmu.edu/~ark/personas/ (CC BY-SA licence).

[2] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019.

[3] V. Sanh, L. Debut, J. Chaumond and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[4] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. EMNLP 2020: System Demonstrations*, 2020.

[5] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019.

[7] M. Sundararajan, A. Taly and Q. Yan, “Axiomatic Attribution for Deep Networks,” in *Proc. 34th International Conference on Machine Learning (ICML)*, 2017.

[8] J. Johnson, M. Douze and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 2021 (arXiv:1702.08734).

[9] K. Sechidis, G. Tsoumakas and I. Vlahavas, “On the Stratification of Multi-Label Data,” in *Proc. ECML PKDD*, 2011.

[10] N. Kokhlikyan et al., “Captum: A unified and generic model interpretability library for PyTorch,” arXiv:2009.07896, 2020.

[11] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[12] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conference (SciPy)*, 2010.

[13] [To be added by the team] Reference documents and cited papers of the existing project “Movie Genre Classification and Cast Recognition using Deep Learning.”